

7/25/23, 2:13 AM

CodeProjectFileEditFindViewToolsEducationHelpRunProject index (stand)Debug

Terminalx pyproject.toml

```
Package name [existing-project]: tree
Version [0.1.0]:
Description []:
Author [Sandipan Dey <sandipan.dey@gmail.com>, n to skip]:
License []:
Compatible Python versions [^3.6]:

Would you like to define your main dependencies interactively? (yes/no) [yes]
You can specify a package in the following forms:
- A single name (requests)
- A name and a constraint (requests<2.23.0)
- A git url (git+https://github.com/python-poetry/poetry.git)
- A git url with a revision (git+https://github.com/python-poetry/poetry.git#develop)
- A file path (../my-package/my-package.whl)
- A directory (../my-package/)
- A url (https://example.com/packages/my-package-0.1.0.tar.gz)

Search for package to add (or leave blank to continue):

Would you like to define your development dependencies interactively? (yes/no) [yes]
Search for package to add (or leave blank to continue):

Generated file

[tool.poetry]
name = "tree"
version = "0.1.0"
description = ""
authors = ["Sandipan Dey <sandipan.dey@gmail.com>"]

[tool.poetry.dependencies]
python = "^3.6"

[tool.poetry.dev-dependencies]

[build-system]
requires = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"

Do you confirm generation? (yes/no) [yes]
codio@cleverlogo-eventcricket:~/workspace/existing-project$ poetry init
A pyproject.toml file with a poetry section already exists.
codio@cleverlogo-eventcricket:~/workspace/existing-project$ poetry update
Creating virtualenv my-script=python3.6 in /home/codio/.cache/pypoetry/virtualenvs
Updating dependencies
Resolving dependencies... (0.1s)
codio@cleverlogo-eventcricket:~/workspace/existing-project$ poetry run python my_script.py
Reporting for duty
codio@cleverlogo-eventcricket:~/workspace/existing-project$
```

Using Poetry with an Existing Project

Initializing a Poetry Project

Assume you started to create a project in Python. You had yet to hear of the benefits of Poetry, so you started to build the project manually. The `existing-project` directory contains a single script called `my_script.py`. Change into this directory and show its contents.

```
cd existing-project && tree
```

You do not need to start over to use Poetry with this project. We are going to initialize our Poetry project with the `init` command.

```
poetry init
```

This will start an interactive process where Poetry will ask you a series of questions about the project. This will form the basis of your `pyproject.toml` file. Anything residing between square brackets will be used as the default; just press enter. If there is nothing between the square brackets, then it is an optional question. Poetry gives you the chance to enter your dependencies (production and development).

```
This command will guide you through creating your pyproject.toml config.
Package name [existing-project]:

Version [0.1.0]:

Description []: "Sample project initializing Poetry"

Author [Your Name your_email@provider.com, n to skip]:

License []:

Compatible Python versions [^3.6]:

Would you like to define your main dependencies interactively? (yes/no)

Would you like to define your development dependencies interactively? (yes/no)
```

Before finishing the initialization process, Poetry will show you the `pyproject.toml` file. The example below did not use the interactive process to add dependencies. This will have to be done later with the `add` command as previously discussed.

```
[tool.poetry]
name = "existing-project"
version = "0.1.0"
description = ""
authors = ["Your Name <your_email@provider.com>"]

[tool.poetry.dependencies]
python = "^3.6"

[tool.poetry.dev-dependencies]

[build-system]
requires = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"

Do you confirm generation? (yes/no)
```

Use the `tree` command to see the contents of the directory.

```
tree
```

Unlike the `new` command, Poetry does not make a full-featured project when using `init`. You do not have `__init__.py` file, a testing directory, etc. However, you do have a `pyproject.toml` file so you can now use Poetry. You can always manually create the missing directories and files if necessary.

```
.
├── my_script.py
├── poetry.lock
└── pyproject.toml
```

Try this variation:

- Create a `poetry.lock` file. Running the `update` command will generate a lock file from the dependencies listed in the `toml` file.

```
poetry update
```

Verify that the newly created file exists by printing the hierarchical structure of the directory.

```
poetry show --tree
```

Running Your Project

You can also run scripts with Poetry as well. Since we are in the `existing-project` directory, we can run the file `my_script`. The very first time you do this, you will see a message about Poetry starting a virtual environment. Every subsequent run will also be in a virtual environment. Run the script several times and it will randomly print one of six different greetings.

```
poetry run python my_script.py
```

▼ Did you notice?

In the previous module we ran scripts using `python3`. With Poetry we do not need to specify the version of Python we want to use. The only version of Python specified in the `pyproject.toml` file is version 3.6. You will not accidentally run the script with version 2 of Python.

```
[tool.poetry.dependencies]
python = "^3.6"
```

Reading Question

Select all of the things that happen when you use the `init` command with Poetry. **Hint**, there is more than one correct answer.

- ☐ Poetry use the `pyproject.toml` file and creates a `poetry.lock` file.
- ☐ Poetry creates subdirectories and a `__init__.py` file.
- ☐ Poetry asks you questions about your project (name, author, etc.).
- ☐ Poetry creates a `pyproject.toml` file.

The `init` command is used when incorporating Poetry into an already existing project. Poetry asks you several questions about the project's metadata. It also creates a `pyproject.toml` file. However, it does not create a lock file or any other subdirectories/files. The `new` command sets up a complete Poetry project. The `init` command just gives you a `toml` file based on the questions it asks.

Check ID

Next